The `myUnorderedListTemplate` variable is defined as a `Func<dynamic, object>`, matching the definition of a templated Razor delegate. Within it, the `item` parameter is assumed to represent a collection of cities. These are iterated over and rendered as an unordered list. The following listing shows how to assign the generation of the list to a Razor template delegate.

Listing 3.11 Defining a templated Razor delegate

```
@{
    Func<dynamic, object> myUnorderedListTemplate = @<ul>
        @foreach (var city in item)
        {
            <li>@city.Name</li>
        }
    </ul>;
}
```

Once the template has been defined, you can pass the data generated in listing 3.6 into it:

```
@myUnorderedListTemplate(cities)
```

This example relies on a `dynamic` input parameter, leading to potential errors that only surface at run time if you misspelled the name of a property, for example, or tried to access members that don't exist. You can use strong typing to confine the types that your template accepts, as shown in the following listing, where the `dynamic` parameter has been replaced with a `List<City>`.

Listing 3.12 Narrowing the data item's type in a templated Razor delegate

```
@{
    Func<List<City>, object> myUnorderedListTemplate = @<ul>
        @foreach (var city in item)
        {
            <li>@city.Name</li>
        }
    </ul>;
}
```

One of the drawbacks of the templated Razor delegate is that it only accepts one argument representing the data item, although there is no restriction on the data